Instituto Superior
de Engenharia
Politécnico de Coimbra

Input Partitioning and Equivalence Class Testing

M.Sc. in Informatics Engineering

Black Box Testing

Equivalence Class Testing

Equivalence Class Testing

- Equivalence Class Testing (ECT) aims to use domain knowledge to create meaningful black box tests. It tries to:
  - ***Reduce test redundancy*** *– Attempts to avoid creating unnecessary tests.*
  - ***Identify domain-specific faults*** *– By using domain information, create tests targeting specific system behavior.*

Equivalence Class Testing

- It operates by partitioning the input domain in one or more dimensions (partitions) of equivalence classes.
  - *Equivalence classes represent inputs that can be considered simliar from a testing perspective, because they elicit a similar system response (or Class of response).*
  - *Therefore, it is sufficient to consider a single representative from each equivalence class, for each partition.*

Equivalence Class Testing

Example:

- *Consider a system that receives the grade of a student and outputs the final assessment (FAILED, PASSED):*
- A *single* input partition can be identified, consisting of two classes:
  - *The first class corresponds to grades in the range 0-9. These correspond to FAILED assessment.*
  - *The second class corresponds to grades in the range 10-20. These correspond to grade that results in a PASSED assessment.*
- All inputs belonging to the same class will be considered equivalent for the purpose of testing.

grade	<10	>=10

Equivalence Class Testing

Example:

- We might also be interested in considering invalid classes, which correspond to unadmissible inputs (e.g. grade <0 and grade>20)
  - *These will be useful if we intend to perform robustness testing.*
- In this case, we would have the following partition and equivalence classes.

	Valid Classes		Invalid Classes	
	FAILED	PASSED	TOO_LOW	TOO_HIGH
grade	0..9	10..20	<0	>20

Equivalence Class Testing

Partitions and inputs

- *A simple (simplistic?) approach is to consider each input variable as a partition.*
- *In this case, we get multiple partitions, one per input.*

```
void f ( int temp, int humidity) {
    if(temp<10)
        System.out.println("Cold");
        else
            if(temp>20)
                System.out.println("Hot");
            else
                System.out.println("Moderate");
    if(humidity<20)
        System.out.println("Dry");
    else
        System.out.println("Damp");
    }
```

Partitions

temp	<10	>=10 and <=20	>20
humidity	<20	>=20	

Equivalence classes

Multiple Partitions

- An algorithm may depend on both the gender and age of a person:
- We can consider two input domain partitions, each one with three Equivalence Classes, reflecting relevant input ranges:
  - **AGE:** CHILD, TEENAGER, ADULT, SENIOR
  - **GENDER:** MALE, FEMALE
- The input must then be characterized accross two input partition dimensions, e.g.: {TEENAGER, MALE}

Equivalence Class Testing

Partitions and inputs

- *Considering a partition to correspond to a single input is simple, convenient, to an extent allows some automatation, but is ultimately limited.*
- *Consider, for example a method that classifies Triangles given the sizes of their sides.*
  - *Not much information is gained by looking at each input as an individual partition, beyond validity.*

Equivalence classes

Partitions	Invalid Input	Valid Input
side_a	<=0	>0
side_b	<=0	>0
side_c	<=0	>0

The Triangle Classification Example

- **Equivalence Classes by example:**
  - *Consider the triangle classification problem and the following test inputs:*
    - {8,8,8} – *An equilateral triangle with side length 8.*
    - {2,2,2} – *An equilateral triangle with side length 2.*
    - {10,10,10} – *An equilateral triangle with side length 10.*
    - {20,20,20} – *An equilateral triangle with side length 20.*

All these inputs will elicit the same system response. It is a reasonable assumption that if the system behaves correctly for one of these inputs, it will work correctly for all the other. Therefore, we can say that all of these inputs belong to the same **Equivalence Class** (e.g. the **Equilateral Triangle** class)

The Triangle Classification Example

- **Possible Equivalence Classes for the Triangle classification example might be:**
  - NOT-A-TRIANGLE
  - Scalene
  - Isosceles
  - Equilateral
- **All inputs are considered to be in (only) one of these classes, and these cover all possibilities.**
- **There is only a single input partition.**

Equivalence Class Identification

- **Equivalence Classes identification is an “art”.**
  - It cannot be automated (even if system internals are known), because it depends on domain-specific considerations.
  - Equivalence Classes sometimes suggest themselves, but are not always obvious.
  - *Creative identification of good Equivalence Classes can result in high quality test cases.*
  - We’ll look at another example soon (at lab class).

Types of Equivalence Class Testing

- When analysis yields multiple input partitions, we can consider four types of Equivalence Testing:

Equivalence Class Testing With Multiple Partitions	Normal (positive only)	Robust (pos. & neg.)
Single Fault Assumption model	***Weak Normal***	***Weak Robust***
No Single Fault model	***Strong Normal***	***Strong Robust***

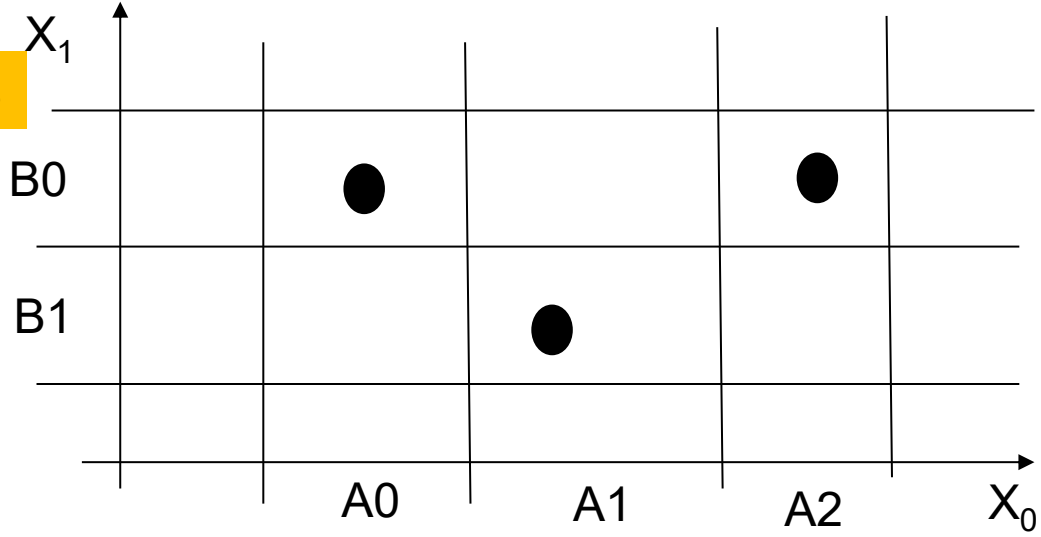
Weak Normal / Each Choice Coverage

- **Weak Normal Equivalence Testing *(aka each-choice cover.)***
  - Based on SFA. Uses one variable from each equivalence class in a test case.

X_0 , X_1 are input partitions

X_0 is [B0 or B1]
X_1 is [A0, A1 or A2]

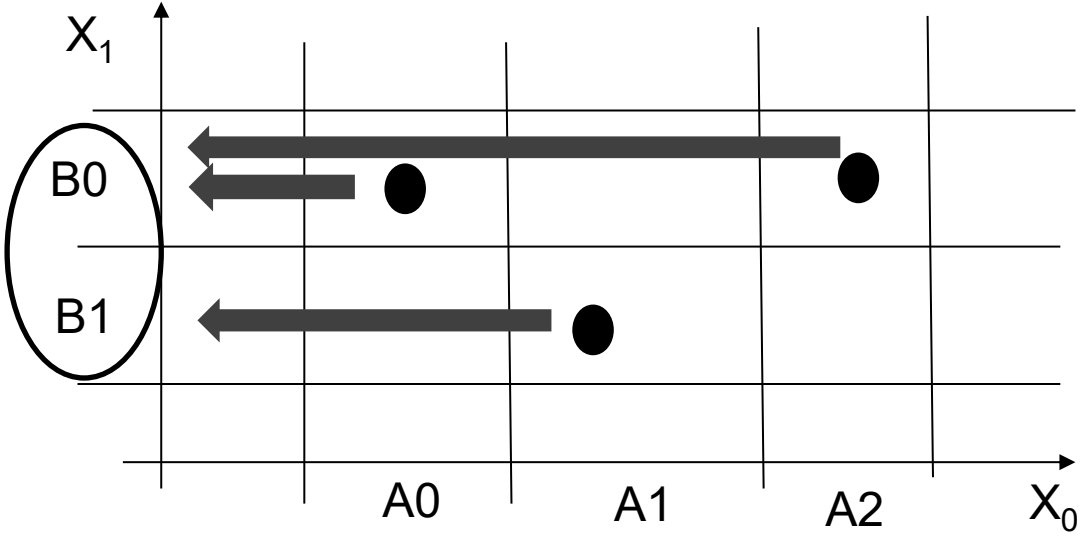
Number of tests:
Max. (Partition size)



Weak Normal / Each Choice Coverage

- **Weak Normal Equivalence Testing *(aka each-choice cover.)***
  - Based on SFA. Uses one variable from each equivalence class in a test case.

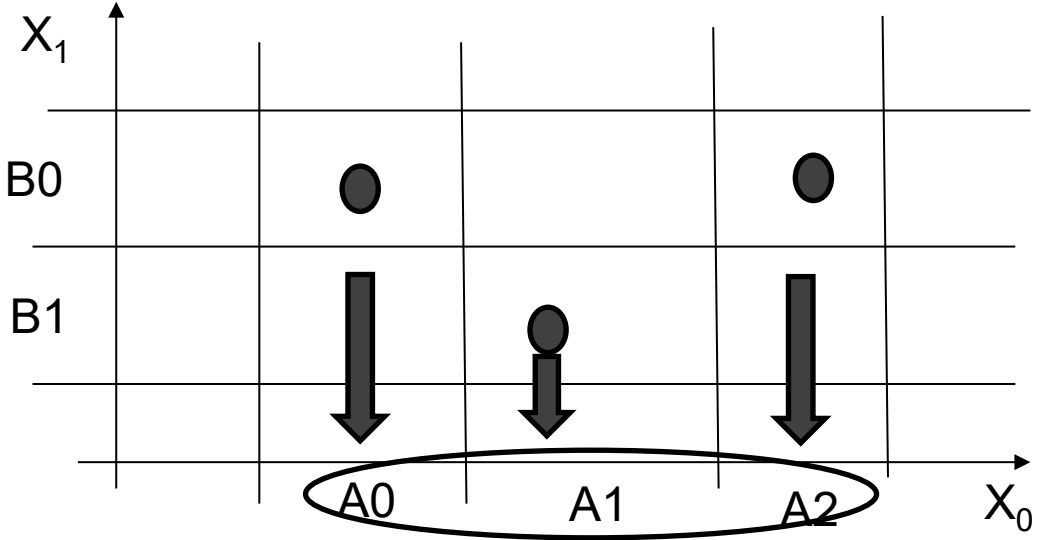
X_0 is [B0 or B1]
X_1 is [A0, A1 or A2]



Weak Normal / Each Choice Coverage

- **Weak Normal Equivalence Testing *(aka each-choice cover.)***
  - Based on SFA. Uses one variable from each equivalence class in a test case.

X_0 is [B0 or B1]
X_1 is [A0, A1 or A2]



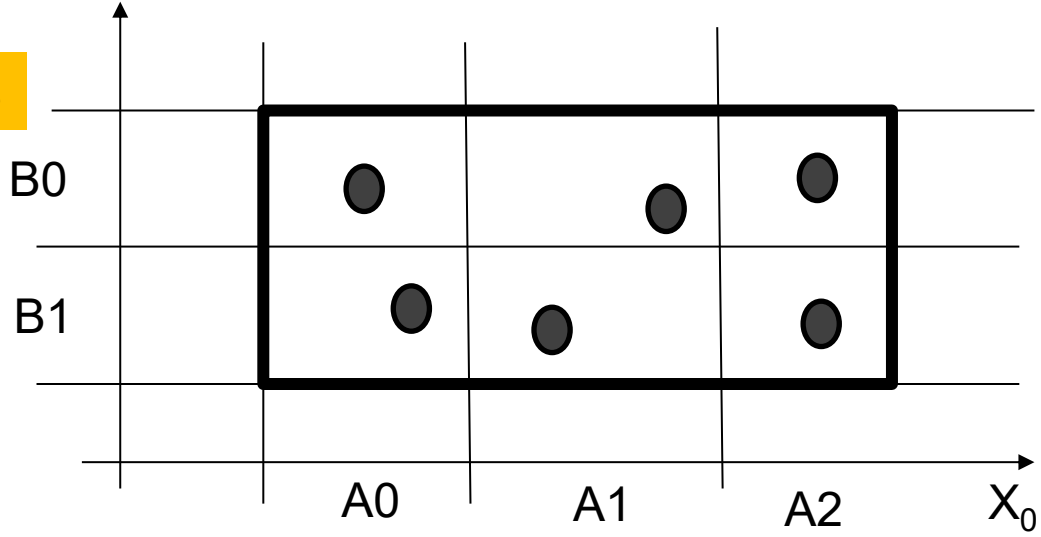
Strong Normal / All-combinations Testing

- **Strong Normal Equivalence Testing (aka *all-combinations coverage*)**
  - Does not assume SFA. Considers all possible equivalence class combinations.

X_0 , X_1 are input partitions

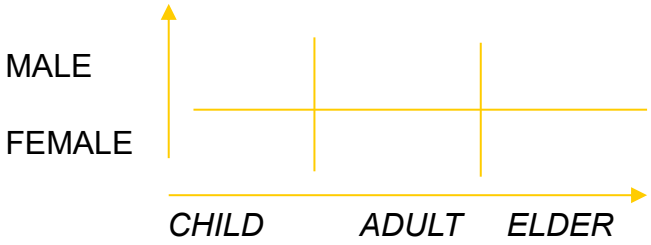
X_0 is [B0 or B1]
X_1 is [A0, A1 or A2]

Number of tests:
Π(Partition sizes)



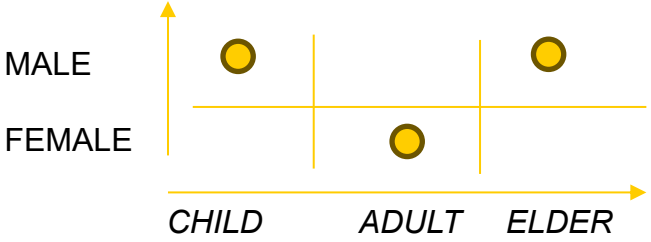
Example

- Consider the Partitions:
  - *IP1={CHILD, ADULT, ELDER}*
  - *IP2={MALE, FEMALE}*
- *This corresponds to the following axis:*



Example

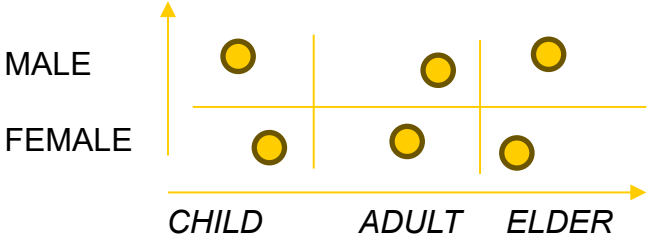
- *Weak Normal Coverage:*
  - *Make sure you have one instance of each class*



Test Inputs:
{Marc, 6 yo, male}
{Anne, 29 yo, female}
{John, 89 yo, male}

Example

- *Strong Normal Coverage:*
  - *Make sure you have all possible combinations of classes each class*



Test Inputs:

{Marc, 6 yo, male}
{Anne, 29 yo, female}
{John, 89 yo, male}
{William, 26yo, male}
{Julia, 7 yo, female}
{Grace, 88 yo, female}

Weak Robust Equivalence Testing

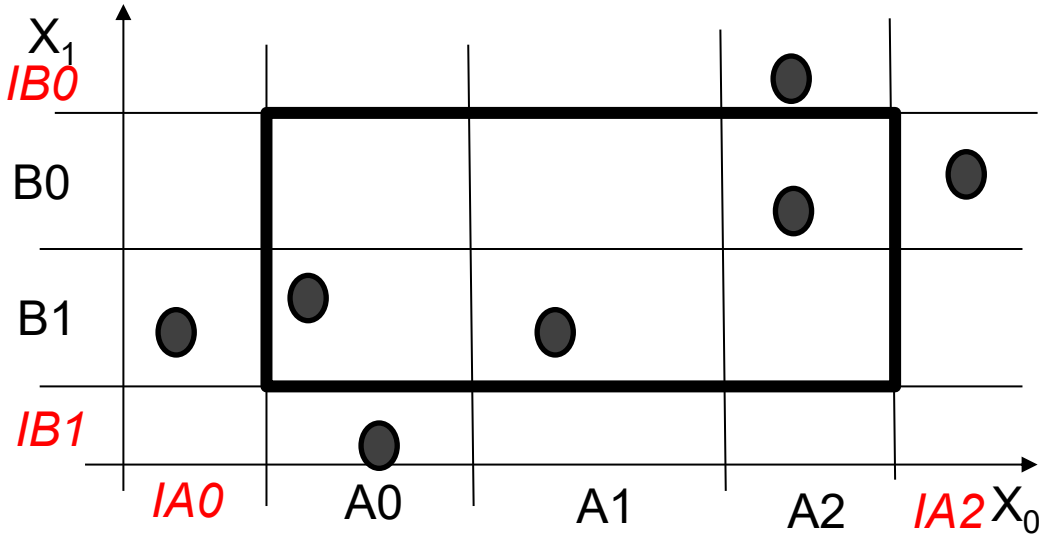
- Weak Robust Equivalence Testing
  - Assumes SFA <u>and</u> the existence of invalid classes.

X_0 , X_1 are input partitions

X_0 is [A0, A1, A2, *IA0,IA2*]
X_1 is [B0, B1, *IB1,IB0*]

Invalid Classes:
IA0,IA1, IB1,IB0

Invalid classes should be tested <u>independently!</u>



Input Partition Dependencies

- If the partitions are not independent, the strong forms may generate invalid test cases.
- As an example, consider a function that receives a date. If the following two partitions are identified:
  - *IP1={MONDAY,TUESDAY,WEDNESDAY,THURSDAY,FRIDAY, SATURDAY, SUNDAY}*
  - *IP2={WEEKEND, WEEKDAY}*

,then the test case {MONDAY, WEEKEND} would be invalid.

Other ECT Criteria

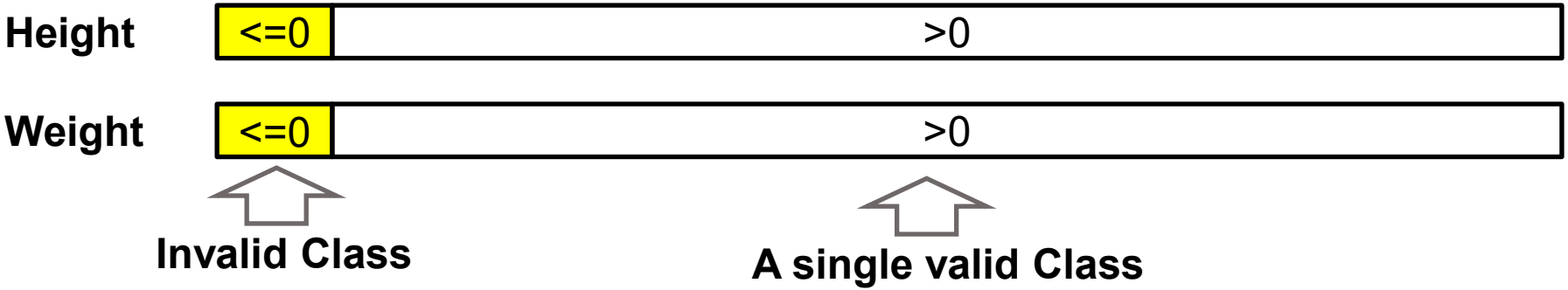
- The weak forms of equivalence class testing (normal or robust) are not as comprehensive as the corresponding strong forms.
- Other intermediate approaches (testing *criteria*) exist e.g:
  - ***Pair-wise combinations*** – all pairs of equivalence classes must be considered.
  - ***T-wise combinations*** – all equivalence classes sets with T elements must be considered.
  - ***Base choice*** – A baseline equivalence set is selected and the tests are created by varying a single dimension at a time.
  - ***Multiple base choice****:* same, but with multiple baselines.

Partitioning other spaces

- Partitioning does not need to (and should not) be restricted to input values only!
- *The same concept applies to inputs, outputs and also internal state.*
- *Choosing the correct application option is fundamental for improving the quality of the generated test set.*
- *The designer should keep in mind all the relevant possibilities.*
- *Let's look at an example.*

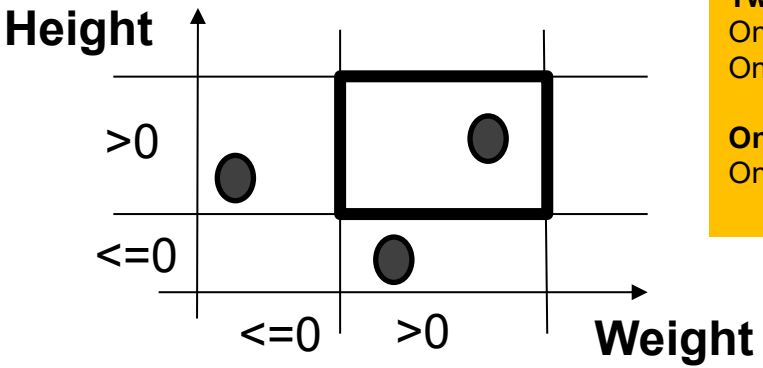
Partitioning other spaces

- Example: Computing the BMI (Body-Mass Index) status.
  - Consider a function that receives the height, weight and computes the status (Underweight, Healthy, Overweight or Obesity)
- *Let's focus on the inputs. The only meaningful partitions we can identify are probably something like this (we could also contemplate arbitrary max values):*



Partitioning other spaces

- Example: Computing the BMI (Body-Mass Index) status.
  - Consider a function that receives the height, weight and computes the status (Underweight, Healthy, Overweight or Obesity)
- *Using this partition, we could define the following three tests (robust testing):*



Two robustness tests:
One test with zero or negative height
One test with zero or negative weight

One positive test:
One test with positive Weight and Height.

Partitioning other spaces

- Example: Computing the BMI (Body-Mass Index) status.
  - Consider a function that receives the height, weight and computes the status (Underweight, Healthy, Overweight or Obesity)
- *The tests obtained when considering the input only are a bit underwhelming and don't appear very good. Let's focus on the output now.*

Status	Underweight	Healthy	Overweight	Obese

One partition, four classes

Partitioning other spaces

- Example: Computing the BMI (Body-Mass Index) status.
  - Consider a function that receives the height, weight and computes the status (Underweight, Healthy, Overweight or Obesity)
- *The tests obtained when considering the input only are a bit underwhelming and don't appear very good. Let's focus on the output now.*

	●	●	●	●
Status	Underweight	Healthy	Overweight	Obese

Four tests are identified, with all possible outcomes being represented in a test. This seems much more sensible…

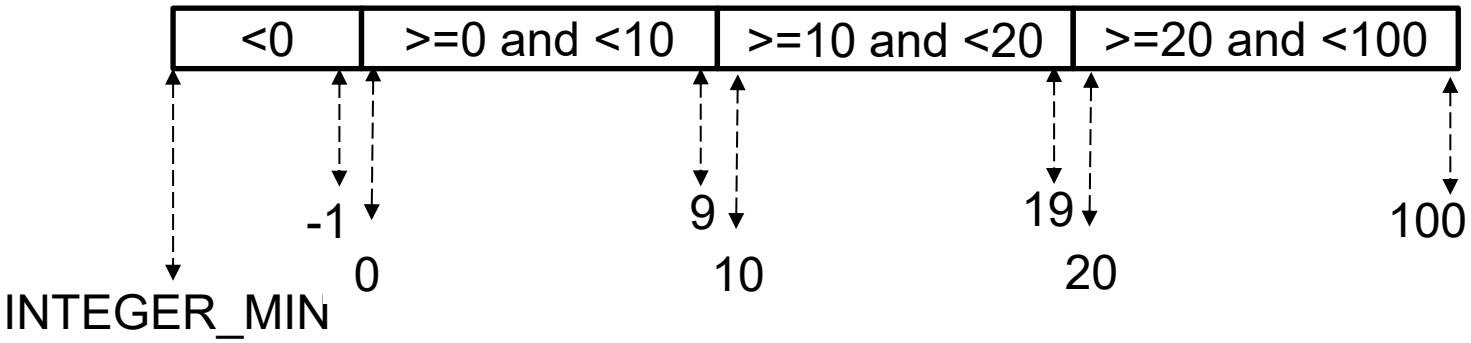
But could we do better than this? Hold that though …

Applying BVA to partition classes

- One major application of partitioning is identifying the system boundaries upon which BVA might be applied:
- Rather than using any arbitrary value from a class, boundary values will be selected instead.
  - *In this case, the boundaries correspond to the class boundaries.*
  - *More than one test per class will be included in the test suite.*
- There are several ways in which BVA might be applied in a partition with multiple classes. According to the ITSQB, we can identify two approaches:
  - ***The 2-value approach:*** *For each class, use the boundary values (max,min) and the elements just above and below the max and min, respectively*
  - ***The 3-value approach:*** *For each class, use the boundary values (max,min) and the elements just above and below max and above and below min.*

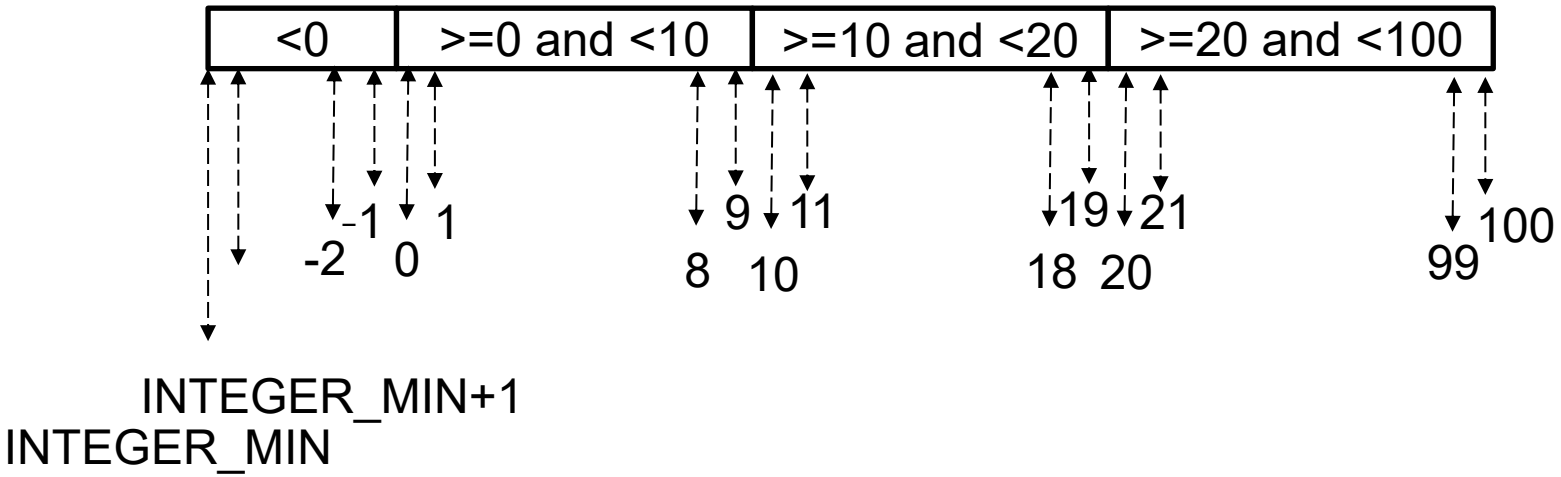
Applying BVA to partition classes

- **The 2-value approach:** Effectively, just includes the boundary values of each class.



Applying BVA to partition classes

- **The 3-value approach:** In this case, some values from inside the classes will also be included.



Partitioning other spaces

- Back to our Example: Computing the BMI (Body-Mass Index) status.
  - Consider a function that receives the height, weight and computes the status (Underweight, Healthy, Overweight or Obesity)
- *Although the tests identified by looking at the output partition seem satisfactory, it is really not possible to perform BVA on these discrete categories.*

Status	Underweight	Healthy	Overweight	Obese

Partitioning other spaces

- Back to our Example: Computing the BMI (Body-Mass Index) status.
- *Let's now consider our knowledge concerning the* <u>*inner state*</u> *of the function.*
- *To do this, we leverage our knowledge that the weight status categories are derived from the computed Body-Mass Index according to the following table:*

Category	BMI (kg/m^2)
Underweight	< 18.5
Normal range	18.5 – 24.9
Overweight	25.0 – 29.9
Obese	≥ 30.0

Partitioning other spaces

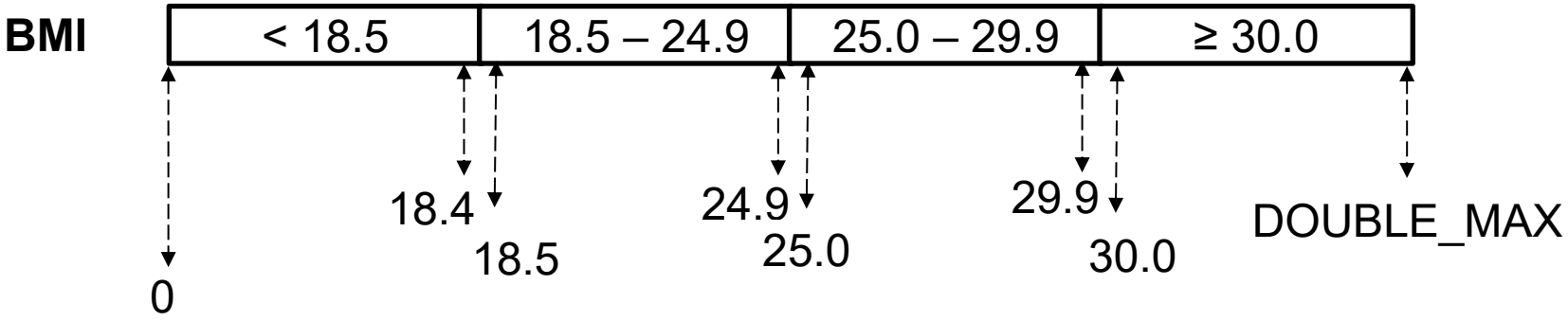
- Back to our Example: Computing the BMI (Body-Mass Index) status.
- *The following partition can be considered instead, based on the internal BMI value:*

Category	BMI (kg/m^2)
Underweight	< 18.5
Normal range	18.5 – 24.9
Overweight	25.0 – 29.9
Obese	≥ 30.0

BMI	< 18.5	18.5 – 24.9	25.0 – 29.9	≥ 30.0

Four tests are also identified, with all outcomes considered. However, using BVA is now an option due to the conti nuous scale.

BMI with 2-value BVA over EC



Four tests are also identified, with all outcomes considered. However, using BVA is now an option due to the conti nuous scale.